

NDNSF: A Data-Centric Service Framework for Secure, Collaborative, and Dynamic Networked Applications

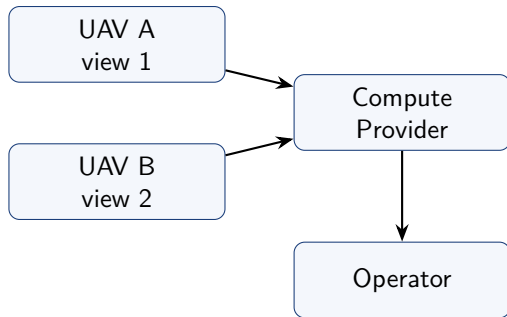
Tianxing Ma

Department of Computer Science
University of Memphis
Advisor: Dr. Lan Wang

PhD Proposal Defense
September 2026

Start with a Task: Several Views, One Result

- ▶ An operator selects a ground area.
- ▶ Nearby UAVs offer suitable views.
- ▶ A compute Provider analyzes the images.
- ▶ The operator receives original images with annotations.



Who can participate now, and which Data may each selected Provider use?

The Task Creates Three Requirements

1. **Discover current willingness:** reachability, sensors, load, and deadlines affect participation.
2. **Authorize before execution:** exposing a service does not grant everyone permission to use it.
3. **Check task inputs:** a valid image signature does not show that its producer was assigned to this task's dependency.

The contribution must explain the mechanism and its cost, not just demonstrate that two UAVs can exchange images.

What NDN Provides—and What It Does Not

NDN building block	Remaining application responsibility
Named Data and signatures [1]	Validate signatures and signing-key relationships under trust rules; check task permissions separately
Interest retrieval and caching [1,26]	Enforce task identity, dependencies, and deadlines; caching is not persistent storage
Dataset synchronization / SVS [2,3]	Discover new names, then retrieve Data; define service-message state transitions
NAC [4] / enhanced NAC-ABE [19]	Use content encryption for read access; separately check service-invocation permissions and state

NAC: Name-based Access Control; ABE: attribute-based encryption; SVS: State Vector Sync.

NDNSF builds on these mechanisms; it does not invent them.

Three Research Questions

- ▶ **RQ1:** How can NDNSF discover willing Providers, keep descriptors confidential from unauthorized readers, and check service permissions before execution, and at what cost?
- ▶ **RQ2:** How can Providers selected at runtime follow task roles and named dependencies, whether Data comes from producers or caches?
- ▶ **RQ3:** When do these mechanisms improve completion, latency, or execution efficiency, and when do their costs outweigh the benefits?

UAV and distributed inference are validation workloads for the same questions; implementing two applications alone does not establish two research contributions.

Related Work Already Provides Important Pieces

Prior work	What the comparison must acknowledge
NFN [6], NFaaS [7]	Named computation and function execution/placement
RICE [9], NSC [10]	Invocation and security mechanisms; NSC validates signed calls using application trust rules
DNMP [12]	Role- and command-constrained signing and verification
CaSCON [15]	Service-composition research; requires a feature-level comparison before claiming a new composition mechanism
gRPC [17,27]	Complete configurations can include discovery/resolver integration, not only fixed addresses

Candidate increment: connect runtime assignment to enforceable object-admission rules.
This is not a claim to be the first NDN service framework.

Contributions and Evidence Are Different Things

Intended contribution	Current evidence boundary
Discovery / authorization design and tradeoffs	Invocation implementation and bounded experiments exist; full alternative lifecycle-cost comparison remains open
ACK-derived plan and local dependency admission	Design and implementation work; complete direct/cache and failure acceptance gates remain open
Cross-case empirical findings	Existing mobility/work-efficiency and small-model results support limited configurations, not all new collaboration claims

Proposal status, September 2026: implemented, locally tested, measured, and fully qualified are not interchangeable. [28]

Participants and Trust Bootstrap

- ▶ User: requests and coordinates the invocation.
- ▶ Provider: offers and executes a service. A node may play both roles.
- ▶ Controller: administers identities and service permissions approved by the organization.
- ▶ Participants use provisioned trust anchors and validation rules for certificates and Controller material.

Unknown runtime candidates are not unregistered authorization identities. The User discovers current participation; the Controller does not choose this task's Providers.

One Invocation, Four Named Data Types

Data	Logical purpose
Request	Describe required service and participation constraints; include protected discovery information and Requester Challenge
ACK	Report positive or negative participation; positive ACK supplies selection information and Provider Challenge
Selection	Assign selected execution; provide input/references, challenge response and protected request-scoped keys
Response	Return status, results or result references; the User still validates them before use

Sync helps discover names; Interests retrieve Data. The Provider checks Selection before execution; a positive ACK alone does not start the task.

Descriptor Before Selection; Input After Selection

Discovery descriptor

Needed to decide whether to participate:
area, deadline, sensor/model and resource requirements.

Application input

Needed to execute the selected task:
images, detailed processing parameters,
or protected input references.

The boundary follows necessity. A location needed to assess coverage belongs in the protected descriptor; an image needed only for processing can wait.

Reason 1: Confidential Discovery Before Selection

- ▶ The User knows the service, but not yet the currently willing Provider identities.
- ▶ An observation area and deadline can reveal intended activity even without the full application input.
- ▶ The User encrypts the descriptor using ABE and the service attribute, without listing Provider identities. [19]

Confidential from parties lacking the required decryption capability. Authorized but unselected Providers can read it; names and traffic patterns remain visible.

Reason 2: Separate Dissemination from Access

- ▶ Nodes may retrieve the same Request ciphertext; only holders of matching decryption keys can read its descriptor.
- ▶ The User encrypts by service attributes, without listing Provider identities. The Controller still authorizes entities and issues keys. [19]
- ▶ Different entities retain overlapping service permissions in individual policy keys, independently of identity-signing credentials.

Dissemination and read access are separate. Combining permissions in one key does not require encrypting all services' content under one shared condition.

How NDNSF Checks Service Permissions

1. The User places a Requester Challenge in ABE-protected Request content.
2. The Provider returns it in ACK Data; the User checks the value against the active request.
3. The Provider encrypts its challenge in ACK content. The User answers in Selection; the Provider checks that response before execution.

Both sides also validate signatures under trust rules and check service permissions and request state. Challenge responses show access to protected values; they do not grant permissions or prevent deliberate sharing.

Compare Authorization and Encryption Separately

Design	Execution authorization	Discovery confidentiality
DNMP-inspired roles [12]	Validator checks role/command signing rules; add invocation-state checks	Add content encryption and key distribution
NDNSF ABE-backed	User/Provider check signatures, challenge responses, service permissions and request state	User encrypts descriptor content with service-level ABE

Trust Schema [16] defines permitted Data-name/signing-key relationships; validators apply its rules. It does not encrypt Content. Encryption and request-state checks are additions to the DNMP-inspired comparison, not capabilities attributed to the original DNMP implementation.

When Another Authorization Design May Be Better

- ▶ Known Providers and simple rules: signed invocation plus recipient encryption may be sufficient.
- ▶ A small, stable receiver group: ordinary group-key management may be attractive.
- ▶ Rich command-argument restrictions: DNMP-style rules make those constraints explicit. [12]
- ▶ Many changing entitlements: measure both policy/key updates and credential/rule updates before claiming a management advantage.

Compare complete combinations of authorization and encryption. ABE-backed authorization is one design choice, not a universally superior solution.

Authorization Lifecycle and Costs

- ▶ The Controller publishes signed permission updates; participants apply them only after retrieval and validation.
- ▶ Request-scoped keys protect input/results. Selection delivers keys to selected Providers; the requesting User retains access.
- ▶ Status checks reject withdrawn permissions; key rotation restricts future encrypted exchanges. Current ABE rotation is Controller-wide. [28]

Lifecycle requirement, not a third advantage: measure policy/key size, challenge cost, propagation and rekeying. No immediate disconnected revocation or recall of old plaintext/keys.

Threat Model: What Is Trusted?

- ▶ Trusted: Controller authorization and the User's planning decision.
- ▶ Untrusted delivery: an object may arrive through a producer, forwarder, cache, or repository and still requires validation.
- ▶ Adversarial cases to test: wrong identities, permissions, challenges, request state, roles, dependencies, or execution identities.
- ▶ Not established by signatures/ABE: honest resource reports, correct inference, truthful images, or resistance to authorized key sharing.

No Byzantine-coordinator consensus claim; no traffic-analysis or instantaneous-revocation guarantee.

RQ2: Generate the Plan from Eligible Positive ACKs

1. Application supplies roles, dependency template, constraints and deadline; the template may be handwritten.
2. Close the ACK collection window and validate eligible positive ACKs.
3. The User applies a replaceable planning policy and signs the resulting role assignments and dependencies.
4. Each Provider validates the plan, derives permitted inputs and rechecks its resources before starting.

The snapshot freezes observed candidates for this decision, not every Provider in the network or their future availability.

A Plan Is More Than a Request ID

Existing request ID	Proposed task-specific plan rules
Identifies the request; requires a validated message	Assigns exact Providers to collection and processing roles
Does not alone specify predecessors	Declares which producer role may supply each named dependency
Does not alone describe key scope	Associates permitted objects, execution identity, and authorization/key scope

Example: within the same request, a service-authorized UAV that was not assigned “view 1” must not satisfy the view-1 dependency.

Plan-to-Data Enforcement: Check Before Advancing

1. Validate the Data signature using the appropriate trust rules.
2. Match the current request, plan and execution identity.
3. Verify that the signer is the assigned predecessor Provider.
4. Check that the object matches the declared edge, permitted name, data format and key scope.
5. Require complete validated input before the dependent computation starts.

Apply the same rules to direct, cached and repository-fetched Data. This is a proposed acceptance contract; full end-to-end qualification remains required.

Failure Handling Needs Explicit State

Event	Required behavior
Late ACK	Do not silently change the committed plan
Duplicate Selection	Follow recorded one-time state; do not implicitly repeat a side effect
Incomplete dependency	Wait within policy/deadline, recover, or fail; do not execute on partial data
Provider replacement	Authorize a new assignment; reject old execution actions and specify permitted result reuse
Cancellation	Prevent late messages from reauthorizing cancelled work

Network re-retrieval is not task re-execution. Compensation is not an atomic transaction or exactly-once physical action.

Proposed UAV Flow: Capture, Analyze and Return

1. Each sensing UAV signs ViewImage Data containing encrypted image content and publishes it in its own authorized namespace.
2. The compute Provider retrieves images over assigned edges and publishes Annotation Data.
3. The compute Provider includes original image names and permitted image keys in its encrypted result for the requester.
4. The requester retrieves the original images and displays the annotations.

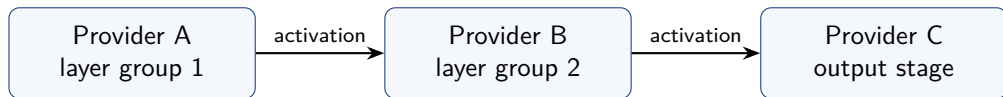
No need to republish the same images as if the compute Provider produced them.
GPS alone does not establish camera visibility or accurate triangulation.

What the UAV Case Must Demonstrate

- ▶ A complete authorized collection–processing–return task before the deadline.
- ▶ Correct handling of unavailable views, delayed images and a failed processing Provider.
- ▶ Reject wrong-role/edge inputs; accept cached images only when they satisfy the same task checks.
- ▶ MiniNDN [22] and PX4 SITL [23] results clearly distinguished from physical-flight evidence.

Higher recognition/localization accuracy is a separate claim requiring ground truth and a single-view control. It is not assumed here.

Proposed DI Flow: Compute Follows Named Dependencies



- ▶ Pipeline: sequential layers/groups; tensor partitioning additionally needs supported collectives.
- ▶ ONNX [25] describes the graph; export alone does not implement distributed execution.
- ▶ Repo [24] stores artifacts; forwarding caches do not guarantee persistence.

The framework contribution is checked runtime assignment and data exchange, not model partitioning itself.

DI State and Evidence Boundaries

- ▶ Model weights, reusable artifacts, request activations and KV state have different lifetimes.
- ▶ Autoregressive decoding requires retained K/V and next-step inputs; resident weights do not remove that state dependency.
- ▶ An in-process test does not establish correct execution across processes or devices.
- ▶ Freeze one supported model/partition first; tensor/hybrid and LLM streaming need their own evidence if claimed.

Earlier small-model measurements do not qualify all current V3/native features. [28]

RQ3: Define the Baselines Before Comparing

- ▶ NSC [10]: named service calls; describe the tested executor, timeout, queue and recovery behavior.
- ▶ gRPC [17]: describe resolver, sequential/parallel policy, deadlines, retry and cancellation.
- ▶ Match Provider count, capacity, task, topology/traces, publication times and measurement windows.
- ▶ Report fixed-list and dynamically discovered configurations as different configurations.

An incomplete baseline cannot establish a limitation of the full framework.
Historical unmatched data remain diagnostics.

Mobility: Overall Completion Depends on Coverage

Ten seeds (62–71), four Providers/system, 2 m/s; mean seed completion rate. [28]

AP radius	NDNSF	gRPC	NSC
50 m	54.57%	53.80%	55.47%
100 m	97.97%	97.97%	98.33%
150 m	100.00%	100.00%	100.00%

These overall controls do not establish NDNSF superiority. The next slide examines the separately registered switching subset, not all requests.

Mobility: Lower Tail Latency in a Defined Subset

Ten seeds; four Providers/system; 100 m, 2 m/s; 1,312 matched requests requiring a Provider switch. [28]

Comparison	Seed-p95 difference	95% CI
NDNSF minus gRPC-SEQ-4	-597 ms	[-992, -200] ms
NDNSF minus NSC-4	-2618 ms	[-2916, -2315] ms

This is a conditional latency result, not general completion-rate superiority. Preserve the registered subset definition, overall coverage controls, and failed requests.

Fewer Execution Starts than the Parallel Baseline

Six-seed, capacity-matched comparison; 1,800 requests per system. [28]

Configuration	Completed requests	Execution starts
NDNSF	1,798 / 1,800	1,800
gRPC-PAR-4	1,800 / 1,800	7,180

NSC-4 also completed 1,800/1,800, with 1.002 starts/request (rounded). No work-efficiency advantage over NSC is claimed.

Fewer starts than gRPC-PAR-4 is not proof of fewer bytes, less CPU time or lower energy. Similar observed completion is not proven equivalence.

A Useful Negative Result: Splitting Can Be Slower

Earlier small Qwen-style ONNX model; ten successful runs per layout. [28]

Layout	Mean latency	p95
Shared-backbone distributed layout	259.4 ms	283.9 ms
Single Provider	186.8 ms	206.3 ms

Do not split merely because several Providers exist. Communication/setup overhead is a candidate explanation, not a causal result established by this measurement.

This campaign does not validate the newer canonical-layer, tensor/hybrid or multi-accelerator design.

Remaining Experiments Follow the Claims

RQ	Required evidence	Outcome and cost
RQ1	Full authorization alternatives; unauthorized call, challenge/replay and withdrawal cases	No unauthorized handler start; discovery delay, key/policy size, status delay and rekey traffic
RQ2	One UAV and one DI graph; wrong role/edge; direct/cache parity; cancellation/replacement	Correct task completion; validation, transfer and recovery cost
RQ3	Matched workload and failure schedules; feasible single-device control; cold/warm state	Completion plus latency, redundant work, bytes, uncertainty, and cases with no advantage

Planned tests are not completed findings. Record all scheduled runs and failure boundaries.

A Bounded Path to Spring 2027

Internal target	Completion gate
September 2026	Agree scope/RQs; freeze one UAV graph and one DI partition; resolve correctness blockers
October–November 2026	Complete registered experiments; freeze evidence and write bounded RQ answers
December 2026	Complete integrated advisor-review draft
January 2027	Committee review and readiness decision; reduce scope or revise timing if core evidence is missing
February 2027	Target defense, conditional on evidence and committee approval

CS guidance: advisor-approved complete draft to committee two weeks before defense. Spring 2027 final ProQuest deadline: March 26, 2027; recheck before filing. [29,30]

What a Final Defense Must Be Able to Claim

- ▶ **RQ1 answer:** when ABE-encrypted discovery and ABE-backed service authorization justify their management and revocation costs.
- ▶ **RQ2 answer:** exactly which task-assignment mistakes are rejected and which recovery behavior is supported.
- ▶ **RQ3 answer:** measured benefits, negative results and deployment limits.
- ▶ Distinguish the candidate's research from shared implementations and reused NDN/AI libraries.

This proposal defines those obligations. It does not label the remaining collaboration evidence as finished.

Discussion: Is This Scope Sufficient and Feasible?

1. Are the three research questions specific enough to support defensible answers?
2. Are the authorization alternatives complete and fairly configured?
3. Are one UAV graph and one supported DI partition sufficient to test the abstraction?
4. Which optional features should be excluded if they delay the core evidence?

NDNSF: runtime service discovery and authorization, connected to explicit named-data execution rules.

Backup: Discovery Configuration Is Not Framework Capability

Provider-transition configuration	Completed
NDNSF, no pre-registered Provider addresses	357 / 357
Static-list NSC/gRPC configurations	0 / 357
Pre-registered baseline variants	357 / 357

This isolates the configured discovery boundary. Resolver/discovery integration can update candidates; it is not proof that gRPC or NSC cannot do so. [27,28]

Backup: Historical Loss Evidence Is Not a New Matched Test

Historical per-link loss	NDNSF completed requests
1%	599 / 600
3%	598 / 600
10% (best recorded run)	539 / 600

A selected run is not an aggregate estimate. Do not combine these numbers with another batch and label the result “matched.” [28]

$1 - (1 - \ell)^4$ assumes four independent equal-loss packet traversals; it is not service failure probability after retries.

References: NDN and Authorization

- [1] L. Zhang et al., “Named Data Networking,” ACM SIGCOMM CCR, 2014.
- [2] T. Li et al., “A Brief Introduction to NDN Dataset Synchronization (NDN Sync),” MILCOM, 2018.
- [3] P. Moll et al., “A Brief Introduction to State Vector Sync,” NDN-0073, rev. 2, 2021.
- [4] Z. Zhang et al., “NAC: Automating Access Control via Named Data,” MILCOM, 2018.
- [12] K. Nichols, “Lessons Learned Building a Secure Network Measurement Framework using Basic NDN,” ACM ICN, 2019.
- [16] Y. Yu et al., “Schematizing Trust in Named Data Networking,” ACM ICN, 2015.
- [19] S. Dulal et al., “Enhancing NAC-ABE to Support Access Control for mHealth Applications and Beyond,” arXiv:2311.07299, 2023.
- [26] NDN Project, “Data Packet,” NDN Packet Format Specification.
docs.named-data.net/NDN-packet-spec/current/data.html (accessed Sept. 2026).

References: Invocation and Composition

- [6] C. Tschudin and M. Sifalakis, “Named Functions and Cached Computations,” IEEE CCNC, 2014.
- [7] M. Krol and I. Psaras, “NFaaS: Named Function as a Service,” ACM ICN, 2017.
- [9] M. Krol et al., “RICE: Remote Method Invocation in ICN,” ACM ICN, 2018.
- [10] D. Meirovitch and L. Zhang, “NSC—Named Service Calls, or a Remote Procedure Call for NDN,” NDN-0074, rev. 1, 2021.
- [15] CaSCON, SMARTCOMP 2025. Primary publication record: RIT, “Barrios presents at IEEE SmartComp.” Full feature-level comparison remains a research obligation.
- [17] gRPC Authors, “Core Concepts, Architecture and Lifecycle,” gRPC documentation.
- [27] gRPC Authors, “Custom Name Resolution,” grpc.io/docs/guides/custom-name-resolution/ (accessed Sept. 2026).

References: Tools, Evidence and Degree Planning

- [22] Mini-NDN, project documentation: minindn.ndn.today.
- [23] PX4, “Software-in-the-Loop Simulation,” PX4 User Guide.
- [24] NDN Project, “Repo Protocol Specification,” [repo-ng](https://repo-ng.com) documentation.
- [25] ONNX, “Open Neural Network Exchange Intermediate Representation Specification.”
- [28] NDNSF, [research-revision-audit.md](#), companion claim/evidence ledger, September 9, 2026. Contains campaign paths and unresolved acceptance gates.
- [29] University of Memphis CS, “Information for Computer Science Ph.D. Students,” [forms_grad_phd_programinfo.pdf](#) (accessed September 9, 2026).
- [30] University of Memphis Graduate School, “Graduation Information,” Spring 2027 calendar (accessed September 9, 2026).